

Audion Disk Tools

Portable-first набор disk/file tools для Windows: аудит, сравнение, синхронизация, backup mirror, сверхширокая фильтрация по типам файлов, мощная архивация, сетевые/SMB-сценарии и native RClone-операции в одном GUI/CLI-инструменте.

Проект больше не завязан на две фиксированные папки. В GUI можно выбрать произвольный **Источник** и произвольную **Назначение**, работать с внешними дисками, сетевыми путями и большими каталогами напрямую, строить список файлов, запускать синхронизацию или собирать архивы без копирования терабайтов внутрь проекта.

Коротко:

- **Источник / Цель** — рабочий маршрут поверх любых доступных Windows/Python папок.
- **Фильтр** — группы и точечные маски расширений для документов, разработки, медиа, архивов, приложений, временных файлов и ручных шаблонов.
- **Операции** — compare, one-way sync, two-way sync, hard mirror, quarantine mirror, Manifest / Diff и копирование по маске.
- **Архивация** — ZIP, 7Z, SFX, TAR, TAR.GZ, TAR.ZSTD и дополнительный TAR.LZ4, если найден `lz4.exe`; шифрование AES-256 для 7Z и SFX; SFX-автоэкстракция; упаковка SFX в отправляемые контейнеры.
- **Трансфер данных** — ROOT-режим наверху: папка, шара, сервер и облако наравне. Один эталон, список машин, пять операций; Robocopy или RClone под каждую пару выбирается сам.
- **Журнал** — живой терминал, отчёты, последний отчёт одной кнопкой, UTF-8/кириллица и ANSI-цвета.

Полная документация

Проект вырос из README в полноценный stack документации:

- `USER_GUIDE_RU.md` - основное пользовательское руководство.
- `USER_GUIDE_EN.md` - английская версия.
- `docs\COMMAND_REFERENCE_RU.md` - полный справочник GUI/CLI команд и параметров.
- `docs\WORKBENCH_AND_GUI_RU.md` - Workbench, терминал, tooltip, кэши и pin/delete UX.
- `docs\SYNC_PROFILES_RU.md` - профили, пресеты и фильтры.
- `docs\ARCHIVE_OPERATIONS_RU.md` - архивы, SFX, шифрование.
- `docs\TRANSFER_RU.md` - трансфер данных: операции, движки, маски, упаковка перед отправкой.
- `docs\RCLONE_OPERATIONS_RU.md` - подключения, хранилище, диагностика, шифрование конфига.
- `docs\RCLONE_PORTABILITY_RU.md` - установка RClone, portable/system config, import/export.

- `docs\MANIFEST_REFERENCE_RU.md` - Manifest / Diff, списки относительных путей Source -> Target и служебные `__CHECKSUMS__.b3`.
- `docs\PDF_EXPORT_RU.md` - сборка Markdown-документации в `docs\PDF`.
- `docs\KNOWN_PITFALLS_RU.md` - типовые риски.
- `docs\SMOKE_TEST_CHECKLIST_RU.md` - проверка после изменений.

В проекте сохранены два основных пользовательских launcher-a:

- `launcher_project.cmd` — главный английский launcher
- `launcher_profiles.cmd` — launcher сохранённых профилей синхронизации

Дополнительно теперь есть:

- `launcher_gui.cmd` — NiceGUI/pywebview GUI-shell поверх существующего CLI
- `launcher_project_ru.cmd` — русская копия главного launcher-a
- `launcher_profiles_ru.cmd` — русская копия launcher-a профилей
- template-owned `builder_main.cmd`
- template-owned `launcher_tools.cmd`

Текущее поведение launcher-ов:

- оба main launcher-a и оба launcher-a профилей поддерживают `FZF` и `CMD fallback`
- GUI сохраняет CLI как источник истины: команды запускаются через `system_core\main.py`, а вывод виден в правом терминале
- в GUI у `ТРАНСФЕР ДАННЫХ`, `ПАПКА В ПАПКУ` и `Маски` есть общий блок расширений: `ВКЛЮЧИТЬ` / `ИСКЛЮЧИТЬ`, закрепляемые наборы, кэш и четырёхстрочная сводка выбранных групп и точечных масок
- в GUI кнопки рабочей панели **Источник**, **Назначение** и **Список** работают с произвольными внешними папками напрямую: большие папки не нужно копировать в `input`
- в GUI `ТРАНСФЕР ДАННЫХ` использует текущий Workbench `Источник` / `Назначение`, saved `remote:path` и парный endpoint builder; команды строятся аргументами subprocess, а в превью показываются с корректными кавычками
- текущие source/target пути при каждом запуске GUI стартуют из проектных `input/output`; выбранные маршруты попадают в локальный `config\path_history.json` до 100 записей, а важные маршруты можно закреплять Pin/Unpin без записи машинных путей в portable-настройки
- GUI может открыть последний человекочитаемый отчёт одной кнопкой в правой панели
- GUI берёт шаблонные палитры из `config\ui_colors.yaml`, а выбранную тему хранит в `config\gui_settings.yaml`; по умолчанию включена `code_dark`
- в шапке GUI есть переключатель темы, RU-подписи укорочены под компактную раскладку, а правую терминальную панель можно менять по ширине
- правый GUI-терминал поддерживает UTF-8/кириллицу, ANSI-цвета, живой построчный вывод и перенос длинных строк без горизонтальной прокрутки
- пути можно задавать вручную или через Windows folder picker

- запуск можно временно переопределять масками имени и расширения файлов, например `*.docx` или `*.docx;*.pdf`

GUI-shell подробно

Запуск GUI:

```
launcher_gui.cmd
```

GUI не заменяет CLI, а превращает его в рабочую панель файлового менеджера. Левая часть окна отвечает за маршрут, сценарий и параметры, правая часть показывает статус, прогресс и живой терминальный журнал. Кнопки действий остаются короткими; подробный смысл живёт в tooltip, README и описаниях операций.

Основные зоны GUI:

- **Workbench** — канонические источник/цель, единый Pin/Unpin, выбор внешней папки или одного файла-источника без копирования, защищённое удаление и список файлов источника.
- **ROOT-действия** — наверху **ТРАНСФЕР ДАННЫХ** (список машин, пять операций, движок под пару выбирается сам), под ним **ПАПКА В ПАПКУ** — прогон аудитора по паре со сверкой BLAKE3, карантин и отчётом. У обоих такая же панель расширений, как у сохранённых профилей: include/exclude, пины, кэш, очистка и сводка выбора.
- **Маски** — ручной режим и генератор glob-строки для внешних программ: выбранные группы и точечные расширения собираются в готовый список вроде `*.pdf, *.docx, *.md`.
- **Архивация** — ROOT-действие для упаковки выбранного источника в выбранную цель: обычные архивы, SFX, шифрование и пост-упаковка SFX.
- **ТРАНСФЕР ДАННЫХ** — ROOT-действие наверху списка: перенос по списку машин с выбором операции, хранилище (объём, дубликаты, суммы, ссылки), подключения (OAuth, S3/R2 по ключам, SFTP, шифрование конфига) и диагностика.
- **Manifest / Diff** — построить отфильтрованный Diff по текущему маршруту Workbench **Источник -> Назначение** и применить его как One-Way или BACKUP-MIRROR; служебные checksum-кнопки всё ещё создают/проверяют `__CHECKSUMS__.b3` для источника/цели.
- **Правая панель** — текущий статус, прогресс, живой stdout/stderr CLI, кнопки **Логи**, **Отчёты**, **Последний**, **Настройки** и разворот журнала.
- **Шапка** — название инструмента, компактный выбор темы и переключатель RU/EN. Ширина терминала сохраняется локально в браузерном профиле GUI.

Операции в GUI:

- **Зеркало** — копирование/обновление, проверка, затем удаление target-only файлов, если ранние фазы успешны. Приёмник становится копией источника и ничего своего не имеет.
- **Односторонняя** — копирует новые и изменённые файлы из **source** в **target**; лишнее в **target** остаётся жить.
- **Двусторонняя** — обменивает отсутствующие или более новые файлы в обе стороны без автоматических удалений.

- **Перенести** — приезжает и пропадает с источника (только в **ТРАНСФЕР ДАННЫХ**).
- **Сверить** — ничего не пишется, только отчёт о расхождении.
- Сухой прогон — переключатель над полями, а не отдельная команда: он действует на выбранную операцию.

Dry-run/preview остаётся полезным диагностическим режимом, но основная безопасность теперь задаётся operation policy и фазным apply.

Что делает проект

Manifest / Diff

Строит отфильтрованный Diff по текущему маршруту Workbench **Источник -> Назначение** и применяет его как One-Way copy/update или BACKUP-MIRROR. Пустой выбор расширений означает все файлы. Служебные checksum-команды всё ещё создают и проверяют **__CHECKSUMS__.b3** для источника или цели, если нужен отдельный integrity-manifest.

Compare

Сравнивает **source** и **target** и пишет отчёты в **output**.

One-way sync

Копирует из **source** в **target** только новые и изменённые файлы.

Backup mirror

Приводит **target** к состоянию **source**. Backup mirror намеренно hard: target-only файлы удаляются после успешных copy/update/verify фаз. **MIRROR_SAFE** остаётся альтернативной policy с карантином.

Full sync two-way

Команда **sync2** докидывает отсутствующие или более новые файлы в обе стороны без автоматических удалений.

Архивация

Создаёт архивы из выбранного источника в выбранную цель. Основная линейка GUI показывает ZIP, 7Z, SFX, TAR, TAR.GZ и TAR.ZSTD при доступном backend; TAR.LZ4 появляется только если найден **lz4.exe**. Шифрование доступно только для 7Z и SFX и всегда даёт AES-256, с шифрованием имён файлов или без; ZIP намеренно оставлен без шифрования, потому что 7-Zip закрывает его устаревшим ZipCrypto, а у семейства TAR шифрования нет вовсе. Неподдерживаемые сочетания в GUI затемняются и объясняют причину. Архивы можно проверять сразу после создания перед возможным удалением исходников. SFX может иметь путь автоэкстракции через Windows-переменные окружения и, при необходимости, дополнительно упаковываться в ZIP/7Z/ZSTD/TAR/GZ контейнер после SFX-компрессии.

ТРАНСФЕР ДАННЫХ

ТРАНСФЕР ДАННЫХ стоит в ROOT первым. Раньше это были два раздела: **ОПЕРАЦИИ ПО СЕТИ** для LAN, SMB и UNC и **ОБЛАЧНЫЕ ОПЕРАЦИИ** для rclone-remotes. Они делили одно занятие по инструменту, а не по задаче, и чтобы

выбрать кнопку, надо было заранее знать, какой инструмент куда дотягивается.

Теперь выбирают операцию — **Односторонняя**, **Зеркало**, **Перенести**, **Двусторонняя**, **Сверить** — и назначение. Назначение может быть списком: один эталон, несколько машин, один запуск. Движок под каждую пару выбирается сам и назван в окне строкой: Robocopy на папки и шары, RClone на облака, двустороннюю синхронизацию и сверку по хешу. Вся раскладка собирается до старта, поэтому невозможная пара видна раньше, чем тронется первый байт.

Сначала упаковать собирает источник в архивы и отправляет их вместо россыпи файлов — на мелких файлах по сети это единственное, что помогает: замерено 24 МБ/с одним файлом против 0.46 МБ/с тремястами мелких.

Пробный прогон есть у всех операций. Потолок удалений (**--max-delete**) доступен и не навязан.

Рядом в том же разделе: **Хранилище** (объём, дубликаты, контрольные суммы, публичные ссылки), **Подключения** (OAuth для Yandex/Drive/OneDrive/Dropbox/Box/Mega, S3-совместимые по ключам, SFTP, шифрование конфига) и **Диагностика**.

Подключения и конфигурация

Для первого запуска откройте **Настройки -> Настройка**: **System** ставит пользовательский RClone в **%LOCALAPPDATA%\Programs\rclone**, **Portable** — в **Tools\rclone**; выбор конфигурации **Portable/System/Custom path**, поле собственного пути и кнопки **Import**, **Export**, **Doctor** находятся в **Configuration**. По умолчанию GUI использует переносимую конфигурацию **config\rclone\rclone.conf** и кэш **tmp\rclone-cache**. Затем используйте **Провайдеры OAuth** для Yandex/Drive/OneDrive/Dropbox/Box/Mega или **Create/update SFTP remote** для серверов. Обычная безопасная отправка строится как **rclone copy** **<Workbench Source> <remote>:<path>**, получение — как **rclone copy <remote>:<path> <Workbench Target>**, а универсальный маршрут — как **rclone copy <SOURCE endpoint> <TARGET endpoint>**.

Видны только команды выбранной группы, а активная подсвечена приглушённым success-цветом. Каждый запуск пишет rclone log в **logs\YYYYMMDD_HHMMSS_rclone_<mode>.log**. Конструктор команды показывается для transfer/route-операций и содержит exe, route, config/cache, source/target, log и полный command preview мелким шрифтом; шаблоны можно закреплять, откреплять и удалять из **config\rclone_command_cache.json**. SFTP **user** и **host/IP** получают отдельный history dropdown в **config\rclone_endpoint_history.json**; OAuth tokens и SFTP password не сохраняются в project JSON/YAML.

Во время transfer/check GUI парсит штатные строки rclone **Transferred**, **Checks**, **Retries**, **Errors** и обновляет верхний progress bar/status: процент, скорость, ETA, checks, retries и errors. Полный поток rclone остаётся в правом терминале и log-файле.

Базовая модель синхронизации

Для one-way сценариев:

- **source** = источник истины
- **target** = приёмник или backup
- копируются только новые и изменённые файлы
- режимы сравнения явные: **quick** = size + mtime, **safe** = hash для same-size неоднозначностей, **strict** = hash same-size кандидатов даже при совпадающем mtime

Это делает главный launcher удобным для backup, export и аккуратных sync-задач.

Как выбрать режим

Сценарий	Команда / GUI	Направление	Удаляет файлы	С чего начинать	Когда использовать
Проверка папки	<code>audit</code>	одна папка	нет	сразу можно запускать	Контроль целостности через <code>__CHECKSUMS__ .b3</code> .
Manifest / Diff	GUI <code>Manifest / Diff</code>	<code>source</code> -> <code>target</code>	зависит от действия применения	выбрать расширения или оставить пусто	Построить отфильтрованный Diff и применить его как One-Way или BACKUP-MIRROR.
Сравнение	<code>compare</code>	<code>source</code> -> <code>target</code>	нет	перед любой серьёзной операцией	Понять разницу между двумя папками без изменений на диске.
Пробный one-way sync	<code>sync --dry-run / One-Way пробно</code>	<code>source</code> -> <code>target</code>	нет	после <code>compare</code>	Посмотреть, какие файлы будут скопированы или обновлены.
Односторонняя	CLI <code>sync</code>	<code>source</code> -> <code>target</code>	нет	после dry-run	Обычный export/copy: новые и изменённые файлы уходят в приёмник.
Preview mirror	<code>backup --preview / BACKUP-MIRROR пробно</code>	<code>source</code> -> <code>target</code>	нет записей	optional diagnostic	Посмотреть кандидаты на copy/update/quarantine/delete.
Backup mirror	<code>backup / BACKUP-MIRROR</code>	<code>source</code> -> <code>target</code>	да, target-only файлы	запускать напрямую, когда это нужно	Hard mirror для контролируемых backup-задач.
Quarantine mirror	<code>backup --operation-policy mirror_safe / BACKUP-MIRROR карантин</code>	<code>source</code> -> <code>target</code>	карантин target-only файлов	optional alternate policy	Mirror с quarantine вместо direct delete.

Сценарий	Команда / GUI	Направление	Удаляет файлы	С чего начинать	Когда использовать
Пробная двусторонняя синхронизация	<code>sync2 --dry-run</code> / <code>Two-Way</code> пробно	в обе стороны	нет	перед <code>sync2</code>	Проверить обмен отсутствующими и более новыми файлами.
Two-way sync	<code>sync2</code> / <code>Two-Way sync</code>	в обе стороны	нет	после dry-run	Свести две рабочие папки без автоматических удалений.
Сохранённый профиль	<code>--pair</code> , только консоль	задано в <code>config\sync_pairs.json</code>	зависит от <code>operation_policy</code>	Compare -> optional preview -> Run	<code>copy_update</code> сохраняет target-only файлы; mirror policies задаются явно.
Копирование по маске	<code>copy_by_mask</code> , <code>--mask-globs</code>	<code>source</code> -> <code>target</code>	нет	dry-run	Разовая выборочная передача, например только <code>*.docx</code> или только офисные документы.

Практическая лестница безопасности:

1. Сначала `compare`, если папки большие или важные.
2. Потом preview/dry-run, если нужен диагностический план.
3. Потом реальный запуск.
4. Для повседневных hard mirror задач держать `mode=safe`; `strict` включать, когда важны same-size/same-mtime расхождения.

Presets и сохранённые pairs

Повторяемые фильтры и jobs лежат в:

- `config\sync_presets.json`
- `config\sync_pairs.json`
- `config\path_history.json` создаётся локально GUI, когда используется история путей
- `config\rclone_endpoint_history.json` создаётся локально GUI для последних SFTP user/host без паролей

Рабочий маршрут сбрасывается на проектные `input/output` при запуске GUI и по кнопке `Сбросить`. История выбора хранится отдельно в локальном `config\path_history.json`: до 100 записей, счётчик частоты, дата последнего использования и Pin/Unpin для закрепления важных маршрутов. Portable `config\gui_settings.yaml` не хранит машинно-специфичные source/target пути.

В UI launcher-а это показывается как `profiles`, но CLI и схема конфига по совместимости используют `pairs`.

Базовый абстрактный профиль для разовых копирований по маске:

- `copy_by_mask`

Примеры backup-профилей по умолчанию работают как hard mirror:

- `dev_backup`
- `docs_backup`
- `graphics_backup`
- `video_backup`
- `audio_backup`
- `arch_backup`
- `apps_backup`

`operation_policy` задаётся явно:

- `copy_update` копирует новые/изменённые файлы и никогда не удаляет и не карантинит target-only файлы.
- `mirror_safe` копирует/обновляет, проверяет, затем переносит target-only файлы в `_audion_quarantine\YYYYMMDD_HHMMSS_microseconds`.
- `mirror_hard` — direct-delete mirror и default для команды `backup`.

Если включены include-фильтры, отчёты пишут `mirror_scope: filtered`; out-of-scope файлы в target игнорируются.

Профиль может нести ещё два необязательных ключа. `anchor: true` требует файл-якорь `.audion-anchor` в обоих корнях до сканирования: смонтированный удалённый корень с уснувшим реер-ом выглядит как существующая пустая папка, и без якоря такой прогон читается как "удалить в приёмнике всё". `note` — свободный текст, видимый в списке профилей GUI и в выводе `main.py pairs`. Подробно в `docs\SYNC_PROFILES_RU.md`.

В комплекте есть категории:

- `DEV`
- `DOCS`
- `VIDEO`
- `AUDIO`
- `GRAPHICS`
- `ARCH`
- `APPS`
- `TEMP`

Фильтр как браузер файлов

GUI использует пресеты из `config\sync_presets.json` как широкий файловый браузер по типам данных. Группы можно комбинировать с ручными масками, а выбранный набор применяется только к текущему запуску:

- документы и текст;
- разработка и конфиги;
- приложения и Windows-файлы;

- архивы и образы;
- аудио, видео и графика;
- временные и служебные файлы;
- любые ручные glob-маски вроде `*.docx`, `*.pdf`, `*.zip;*.7z`.

В верхней сводке расширений видно отдельно выбранные группы и точечные маски для **ВКЛЮЧИТЬ** и **ИСКЛЮЧИТЬ**. Исключения вычитаются из итогового списка и уходят в CLI как `--exclude-globs`.

Это делает один и тот же маршрут пригодным для точечного копирования, бэкапа, отчёта, аудита или архивации без смены рабочей папки.

Основные точки входа

GUI-shell

```
launcher_gui.cmd
```

Главный launcher

```
launcher_project.cmd
```

Русский launcher

```
launcher_project_ru.cmd
```

Launcher профилей

```
launcher_profiles.cmd
```

Русский launcher профилей

```
launcher_profiles_ru.cmd
```

Builder

```
builder_main.cmd
```

Сервисные и release-инструменты

```
launcher_tools.cmd
```

Прямые CLI-примеры

Создать manifest:

```
runtime\python.exe system_core\main.py manifest --root "D:\Media" --project-root "%CD%"
```

Проверить manifest:

```
runtime\python.exe system_core\main.py verify --root "D:\Media"
```

Показать сохранённые pairs:

```
runtime\python.exe system_core\main.py pairs
```

Пометить смонтированный удалённый корень якорем, чтобы отключившийся монтаж не читался как пустая папка:

```
runtime\python.exe system_core\main.py anchor "N:\Projects"
```

Сравнить сохранённую pair:

```
runtime\python.exe system_core\main.py compare --pair "docs_backup"
```

Запустить сохранённую backup pair с её настроенной policy:

```
runtime\python.exe system_core\main.py backup --pair "docs_backup"
```

Preview для backup pair:

```
runtime\python.exe system_core\main.py backup --pair "docs_backup" --dry-run
```

Запустить точное зеркало со штатным режимом сравнения **safe**:

```
runtime\python.exe system_core\main.py backup --source "D:\Work" --target "X:\Backup" --mode safe
```

Запустить то же точное зеркало со строгим сравнением **strict**:

```
runtime\python.exe system_core\main.py backup --source "D:\Work" --target "X:\Backup" --mode strict
```

Запустить full sync two-way вручную:

```
runtime\python.exe system_core\main.py sync2 --source "D:\Notes" --target "X:\Notes" --mode safe --dry-run
```

Exit codes: apply-команды возвращают `0` только когда `errors + conflicts == 0`; реальные `sync2` конфликты не считаются чистым успехом. Preview показывает тот же conflict/error summary в JSON.

Карантин чистится вручную: сначала проверьте `_audion_quarantine\...` и quarantine report, затем удалите старые quarantine-папки.

Текущая структура проекта

- `config\` — пресеты и сохранённые pair-конфиги
- `config\ui_colors.yaml` — палитры, CSS-токены и темы GUI
- `config\gui_settings.yaml` — стартовый язык, выбранная тема и GUI-only настройки
- `install\` — build, install, verify и release-скрипты
- `docs\` — подробные пользовательские и проектные заметки, включая GUI-гайд
- `system_core\` — Python-ядро и внутренние helper-скрипты
- `system_core\license\` — шаблонные release licensing tools
- `runtime\` — portable Python runtime
- `wheelhouse\` — офлайн-кэш wheels
- `input\` — входная зона пользователя
- `output\` — отчёты и summary
- `logs\` — логи выполнения
- `report\` — GUI run-артефакты и machine-readable отчёты
- `workspace\` — управляемая рабочая зона GUI
- `release\` — release-архивы
- `GitHub\` — публикационная документация
- `licenses\` — third-party notices для релизной упаковки
- `._runtime\` — temp-файлы launcher-слоя

Cleanup и init folders

`install\init_folders.cmd` создаёт проектную структуру для GUI/CLI: `input`, `output`, `logs`, `report`, `workspace`, `data`, `runtime`, `wheelhouse`, `release`, `._runtime`, `system_core`, `install` и `licenses`.

`cleanup_project.cmd` — это source-cleanup для подготовки чистой portable-папки. Он оставляет исходники, документацию, постоянные конфиги, sync-профили и лицензии, но очищает управляемые `input/output/logs/report/workspace/data`, runtime/build payload, временные папки и Python-кэши. Скрипт проверяет маркеры проекта и отказывается работать вне корня Audion Disk Auditor.

Текущая temp-схема launcher-ов

Главные launcher-ы теперь используют фиксированные temp-файлы в `._runtime\`:

- EN: `project_menu_en*`, `project_path_mode_en*`

- RU: `project_menu_ru*`, `project_path_mode_ru*`
- profiles EN: `profiles_menu_en*`, `profiles_path_mode_en*`, `profiles_pick_en*`
- profiles RU: `profiles_menu_ru*`, `profiles_path_mode_ru*`, `profiles_pick_ru*`
- builder/tools: шаблонные `builder_menu*`, `tools_menu*`

Старое split-поколение `*_fzf.cmd` выводится из эксплуатации в пользу единой шаблонной схемы launcher-a.

Что подтверждено проверкой

На реальных папках в текущем прогоне подтверждено:

- `launcher_project.cmd`, `launcher_project_ru.cmd`, `launcher_profiles.cmd` и `launcher_profiles_ru.cmd` стартуют и в `FZF`, и в `CMD fallback`
- `compare`, `sync`, `backup` и `sync2` проверены на реальных исходной и целевой папках
- профиль `copy_by_mask` с маской `*.docx` реально скопировал 29 DOCX в реальную целевую папку
- `manifest` создал реальный файл `__CHECKSUMS__.b3` на 2478 записей
- `verify` успешно проверил этот manifest с результатом `ok: true`

Интерактивной веткой остаётся:

- Windows folder picker через `system_core\Pick-Folder.ps1`, потому что он требует реального GUI-выбора

Важное ограничение

Основной файловый CLI работает с путями, которые Windows или Python видят как обычные директории:

- локальные диски
- внешние диски
- UNC-сетевые пути
- смонтированные сетевые папки
- локальные папки облачных клиентов
- смонтированные remote, например через `rclone mount`

Для cloud API и native remotes используйте GUI-раздел **ТРАНСФЕР ДАННЫХ**. Он работает через rclone backend-ы, по умолчанию использует portable `config\rclone\rclone.conf`, и не хранит OAuth tokens или содержимое `rclone.conf` в JSON/YAML проекта.

Практическая рекомендация

Для повседневных Backup mirror задач лучше держать `operation_policy=mirror_hard` и `mode=safe`. Старый ключ `dry_run_default` игнорируется с deprecation warning; используйте явный `operation_policy`.

Канонические названия Workbench

Workbench использует единый публичный словарь Audion Image Tools во всех проектах. Кнопки всегда расположены и называются одинаково: **Источник**, **Добавить файл...**, **Назначение**, **Сбросить**, **Удалить**, **Список**.

Сбросить возвращает проектные **input/output** и не удаляет файлы; **Удалить** очищает текущие **Источник** и **Назначение** только после подтверждения. В английском интерфейсе точные названия: **Source**, **Add file...**, **Target**, **Reset**, **Delete**, **List**. Варианты **Цель**, **Очистить**, **Destination** и **Clear** для этих элементов Workbench не используются.